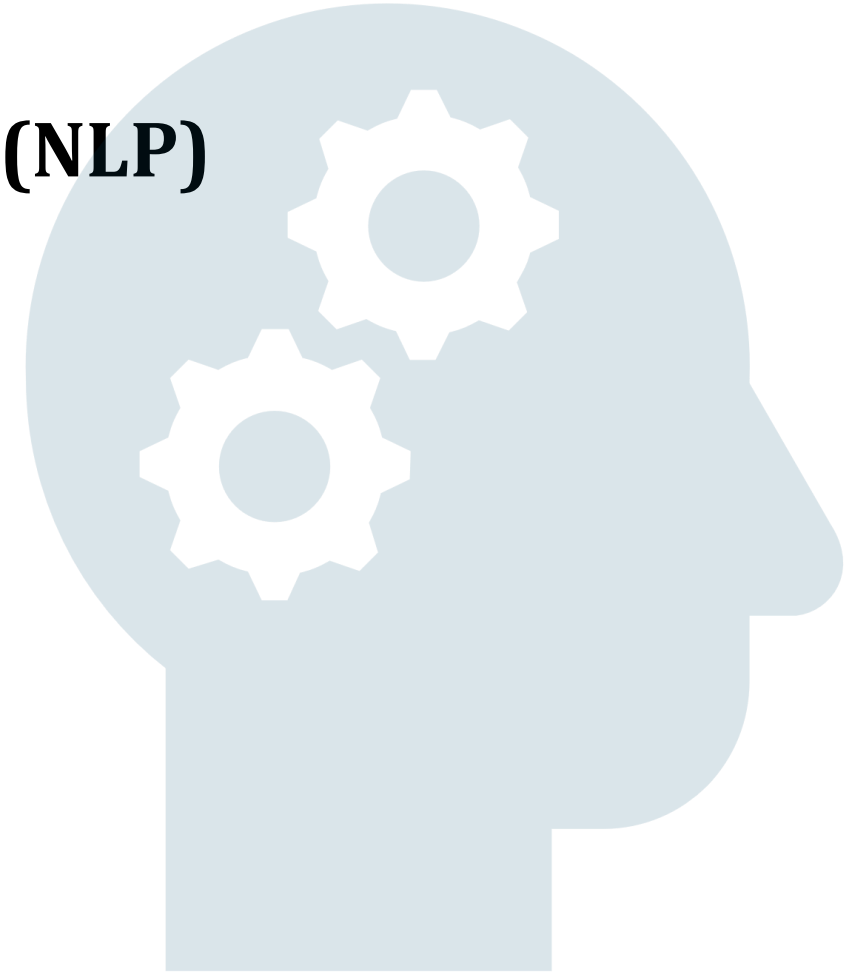
Natural Language Processing (NLP)

Chapter # 2:

Words Embeddings using Dense Vector Representations



Scope

- Word embedding
- Dense Vector Representations
- Properties of Dense Vector Representations
- Methods of Training Word Embeddings
  - 1.Word2Vec
  - 2.GloVe
- Comparison Word2Vec Vs GloVe
- FastText Embedding

Word embedding

Word embedding is a technique in NLP used to **numerically** represent words in a **continuous, dense vector space**, where similar words (in meaning or context) are located close to each other. This is a key concept that enables machines to understand the semantic relationships and syntactic properties of words in a text

Embedding..

- Words will be converted into a numerical format for machine learning models to process them.
- Unlike traditional methods (e.g., one-hot encoding), word embeddings capture relationships between words in a compact, efficient, and meaningful way.

E.g.

king = [0.25, 0.85, 0.75, ...]
queen = [0.26, 0.84, 0.76, ...]

Cat = [0.14, 0.60,0.91…]

Dense Vector Representations

A word is represented as a compact vector with a small number of dimensions (e.g., 50, 100, or 300). Each dimension encodes meaningful information about the word's semantic and syntactic properties.

Dense Vector Vs Traditional Sparse Representations:

Sparse Vectors (e.g. One-Hot Encoding):

- High-dimensional (equal to the vocabulary size).
- Each word is represented as a binary vector with a single 1 and the rest 0
- No semantic information or relationships captured. (e.g. Bank vs bank(Financial))

Dense Vectors (Word Embeddings):

- Low-dimensional (e.g., 300 dimensions for Word2Vec).
- Encodes rich semantic and syntactic relationships.

Properties of Dense Vector Representations

1. Semantic Similarity

Words with similar meanings are close in the vector space.

Example: "king" and "queen" have vectors close to each other.

2. Syntactic Similarity

Words with similar syntactic roles (e.g., verbs) are clustered together.

3. Arithmetic Operations (Vector)

Vector arithmetic can reveal semantic relationships:

$\text{king} - \text{man} + \text{woman} \approx \text{queen}$

Methods of Training Word Embeddings

Word embeddings are typically learned by training models on large text corpora using techniques like:

1. Word2Vec (Skip-gram and CBOW):

Learns embeddings by predicting the context of a word (Skip-gram) or the word given its context (CBOW).

2. GloVe (Global Vectors):

Combines local and global co-occurrence statistics to learn embeddings.

3. FastText:

Extends Word2Vec by considering subword information, handling rare and out-of-vocabulary words better.

4. Pre-trained Word Embeddings (Contextual Word Embeddings)

ELMo, BERT, GPT: Generate dynamic embeddings that vary depending on the word's context in a sentence.

Word2Vec

- Introduced by Tomas Mikolov et al. (2013) from Google
- Trained with a **shallow Neural network** on a large text of corpora
- low-dimensional, continuous vector representations (word embeddings) for words with context in a large corpus of text.
- dense and semantic-rich embeddings

Distributed Representations: Instead of one-hot encoding (sparse and high-dimensional), Word2Vec generates dense, continuous-valued vectors that capture semantic similarity.

Context-Based Learning: Words that appear in similar contexts have similar vector representations.

Word2Vec – Architectures

Continuous Bag of Words (CBOW):

Goal: Predict the target word given the context words.

Mechanism:
- Context words are input to the network.
- The model predicts the center word (target).

Example:
- Input: ["the", "brown", "fox", "over", "the"]
- Output: "jumps"

Advantages:
- Faster to train than Skip-gram.
- Performs well for frequent words.

Skip-gram Model:

Goal: Predict context words given the target word.

Mechanism:
- The target word is input to the network.
- The model predicts surrounding context words.

Example:
- Input: "jumps"
- Output: ["the", "brown", "fox", "over", "the"]

Advantages:
- Performs well for infrequent words.
- Captures complex relationships.

Usage :

- Sentiment analysis
- Machine translation
- Document similarity

Word2Vec – Mathematical Representation

- Maximize the probability of observing context words C given the target word w_t:

$$P(C|w_t) = \prod_{w_c \in C} P(w_c|w_t)$$

- The probability $P(w_c|w_t)$ is computed using softmax:

$$P(w_c|w_t) = \frac{\exp(\vec{w}_t \cdot \vec{w}_c)}{\sum_{w \in V} \exp(\vec{w}_t \cdot \vec{w})}$$

- $\vec{w}_t$ and $\vec{w}_c$ are the vector embeddings for w_t and w_c.

GloVe (Global Vectors for Word Representation)

Concept:

- Statistical alternative to Word2Vec
- Focuses on **word co-occurrence statistics** in a corpus.
- Utilizes a co-occurrence matrix where each entry represents how often two words appear together in a given window.

Co-occurrence Matrix Construction:

- Each entry X_{ij} in the matrix represents how frequently word i appears in the context of word j.

Training Objective:

- Minimize the difference between the dot product of word vectors and the logarithm of their co-occurrence frequency:

$$J = \sum_{i,j} f(X_{ij}) \left(\vec{w}_i \cdot \vec{w}_j + b_i + b_j - \log(X_{ij})\right)^2$$

  - $f(X_{ij})$ is a weighting function to handle rare and frequent word pairs.

Comparison Word2Vec Vs GloVe

Feature	Word2Vec	GloVe
Learning Approach	Predicts context using neural networks	Factorizes co-occurrence matrix
Context Focus	Local (within a sliding window)	Global (entire corpus)
Training Speed	Faster for large corpora	Slower for large corpora
Memory Usage	Lower	Higher due to co-occurrence matrix
Interpretability	Less interpretable	Easier to interpret matrix factors

FastText Embedding

- Developed by Facebook's AI Research (FAIR) lab
- built on word2vec by introducing an improved representation of words using subword (character n-gram) E.g. "apple" >> represented as the n-grams <ap, app, ppl, ple, le>, and so on.
- Good for out-of-vocabulary (OOV) words more effectively
- Capture semantic similarities better
- Pre-trained Models: FastText offers pre-trained embeddings for many languages trained on large datasets like Wikipedia
- FastText embeddings are computationally efficient and scalable, capable of training on large datasets quickly.

Pre-trained Embeddings in NLP

Pre-trained embeddings in NLP are vector representations of words or sub words that are generated by training on a large corpus of text data. These embeddings capture the semantic and syntactic meanings of words based on their contexts in the training data

- **Word2Vec**
- **GloVe**
- **FastText**
- **ELMo**
- **BERT**
- **GPT(s)**

Pre-trained Embeddings

Usage :

- Feature Extraction
- Fine-tuning
- Transfer Learning (Knowledge)

Applications :

- Text Classification
- Machine Translation
- Named Entity Recognition (NER)

Products

Building NLP applications, like ChatGPT, Gemini, CoPilot

Advantage of Pre-trained Embeddings

- Semantic Understanding:
- Efficient Training:
- Domain Adaptation:
- Handling Data Scarcity:
- Better Generalization (Due to wide variety of linguistic)
- Improved Contextualization (in Transformer-based Models): (e.g. Bank)
- Adaptability Across Languages: (Language translations)

Disadvantage of Pre-trained Embeddings

- Domain Mismatch: if the trained domain is significantly different than target domain
- Fixed Embeddings (in Non-Transformer Models): Older embeddings (e.g., Word2Vec, GloVe) are static and do not adapt to context, unlike transformers.
- Modern pre-trained models (E.g.BERT and GPT) require significant computational resources to train and fine-tune.
- Bias in Training Data: Embeddings may reflect and propagate biases present in the training data.